

Actividad de Reflexión - Patrón Singleton

1. ¿Qué problema busca solucionar el patrón Singleton?

El patrón **Singleton** busca garantizar que una clase tenga una única instancia durante toda la ejecución de la aplicación, proporcionando además un punto de acceso global a dicha instancia. Esto evita la creación innecesaria de múltiples objetos que representan el mismo recurso o servicio.

2. ¿Por qué es importante tener una única instancia?

Tener una única instancia permite mantener un estado global consistente en toda la aplicación. Por ejemplo, si cada controlador tuviera su propio contador de consultas, no sería posible conocer el número total de consultas realizadas en el sistema. Al compartir una sola instancia, toda la aplicación trabaja con la misma información.

3. ¿Qué características identifican a una clase Singleton?

Una implementación tradicional del patrón Singleton en Java presenta las siguientes características:

- Constructor privado para impedir la creación de objetos desde otras clases.
- Variable estática encargada de almacenar la única instancia de la clase.
- Método estático `getInstance()` que devuelve dicha instancia.

En **Spring Boot**, la implementación es diferente, ya que el contenedor de inversión de control (IoC) administra automáticamente la instancia única de los componentes anotados con `@Service`, `@Component`, `@Repository` o `@Controller`.

4. ¿Cuál es la diferencia entre múltiples instancias y una única instancia?

Múltiples instancias

- Cada objeto mantiene su propio estado.
- No existe sincronización entre los datos de cada instancia.
- Puede generar inconsistencias cuando se requiere información compartida.

Única instancia

- Todos los componentes utilizan el mismo objeto.
- El estado es compartido y consistente en toda la aplicación.
- La información permanece sincronizada globalmente.

5. ¿Cómo implementa Spring Boot el patrón Singleton?

Spring Boot crea automáticamente una única instancia de cada **Bean** registrado en el contenedor de IoC. Cuando una clase está anotada con `@Service`, `@Component`, `@Repository` o `@Controller`, Spring reutiliza esa misma instancia cada vez que es inyectada mediante la inyección de dependencias.

6. ¿Cuáles son las ventajas de Spring frente a una implementación manual?

Implementar Singleton mediante Spring ofrece diversas ventajas:

- **Menos código:** No es necesario crear constructores privados ni métodos estáticos.
- **Mayor mantenibilidad:** Spring administra automáticamente el ciclo de vida de los objetos.
- **Facilidad para realizar pruebas:** Es posible reemplazar implementaciones reales por objetos simulados (*mocks*) durante las pruebas.
- **Mayor seguridad:** El contenedor garantiza que exista una única instancia del Bean.

7. ¿Cuándo es recomendable utilizar el patrón Singleton?

El patrón Singleton resulta útil en situaciones donde se requiere compartir un mismo recurso o estado entre toda la aplicación, por ejemplo:

- Servicios de configuración global.
- Sistemas de registro de eventos (logs) y auditoría.
- Pools de conexiones a bases de datos o servicios externos.
- Sistemas de caché compartida.
- Contadores y métricas globales.
- Gestores de sesiones o de usuarios autenticados.